

등교시간 예측 서비스 ‘나가자’ (Nagaja - School Departure Time Prediction Service)

본 보고서를 2026년도 Project [등교시간 예측 서비스
‘나가자’]의
최종보고서로 제출합니다.

학부(전공)/학과: 응용소프트웨어공학과

담당(지도)교수: 김삼문

팀명: 가드

제출일: 2026년 06월 12일

팀 장 : 허재경 (20212999)

팀원1 : 곽우석 (20213021)

팀원2 : 김종호 (20200372)

목 차

I. 프로젝트 배경 및 목표.....	1
II. 프로젝트 추진 내용.....	3
1. 프로젝트 추진 일정.....	3
2. 프로젝트 주요 추진 내용.....	4
3. 주요 시행착오 및 해결과정.....	5
III. 프로젝트 추진 결과.....	6
1. 프로젝트 추진 결과물.....	6
2. 프로젝트 결과물 활용 방안 및 기대효과.....	14
IV. 참고문헌.....	15
[부록].....	16

I. 프로젝트 배경 및 목표

1. 프로젝트 배경

가. 현재 사회적 동향 및 기술 상황

최근 바쁜 현대인에게 시간 관리는 삶의 질을 결정하는 중요한 요소로 자리 잡고 있다. 기술의 발전으로 실시간 교통 정보와 날씨 데이터가 API 형태로 개방되면서, 이를 활용한 도착 예정 시간 안내 서비스가 보편화되었다. 그러나 기존 서비스들은 '현재 시점에서 목적지까지의 소요 시간'만을 제공할 뿐, 사용자가 실제로 집에서 나가기 위해 준비하는 시간이나 개인별 이동 습관을 충분히 반영하지 못한다는 한계가 있다.

특히 대학생은 매일 고정된 수업 시작 시각에 맞춰 등교해야 하지만, 교통 혼잡·날씨 변화·개인별 준비 시간 차이 등 다양한 변수로 인해 시각이 빈번하게 발생한다. 기존의 단순 알람 앱은 고정된 시각에만 알림을 제공하여 실시간 상황 변화를 반영하지 못한다.

나. 기존 문제점 분석 및 새로운 요구사항

- **개인별 생활 패턴 미반영** : 기존 지도 서비스는 도로 상황만 고려하며, 세면·옷 입기 등 등교를 위한 '준비 시간'은 계산에서 제외된다.
- **실제 행동 데이터의 부재** : 소프트웨어 기반 서비스는 사용자가 정확히 언제 출발했는지에 대한 실측 데이터를 수집하기 어려워 예측 정확도에 한계가 있다.
- **유동적 알림 부족** : 갑작스러운 폭우·사고로 인한 교통 체증 발생 시, 사용자가 평소보다 일찍 일어날 수 있도록 유동적으로 대응하는 맞춤형 알림에 대한 요구가 높다.

다. 프로젝트의 필요성

이러한 한계를 극복하기 위해 사용자의 실제 등교 패턴과 외부 환경(교통·날씨)을 결합한 지능형 예측 서비스가 필요하다. 특히 모바일 앱과 IoT 기반 물리 알람 시계를 연동함으로써 사용자의 실제 기상·출발 데이터를 자동으로 수집하고, 이를 반영하여 개인 맞춤형 출발 시각을 제시하는 서비스가 요구된다.

본 프로젝트는 이러한 필요성에 따라 '지금 준비를 시작해야 할 시각'과 '마지막 출발 가능 시각'을 동적으로 산출하여 안내하는 시스템을 설계·구현하였다.

2. 프로젝트 목표

사용자의 행동 데이터와 실시간 환경 데이터를 종합 분석하여 지각 가능성을 최소화하고, 개인별 맞춤형 준비 시작·출발 시각을 안내하는 IoT 연동 지능형 등교 지원 서비스 ‘나가자’를 개발하는 것을 목표로 한다.

가. 공학적 목표(정량적)

- 기존 도착 예정 시간 대비 ‘실제 도착 시간’ 오차율을 15% 이내로 개선한다.
- IoT 기기(Wi-Fi 연결 감지 등)를 활용한 출발 시각 자동 기록 성공률 95% 이상을 확보한다.
- 도착 시간 예측 오차 ± 3 분 이내를 목표로 시간 계산 엔진을 구현한다.
- 외부 환경 변화(폭우·사고 등) 감지 시 사용자에게 가변적 알람을 제공한다.

나. 비공학적 목표

- 대학생의 아침 시간 부담과 지각에 대한 심리적 불안감을 감소시키는 라이프스타일 어시스턴트로서의 실용성을 확보한다.
- 정확한 숫자보다 ‘현재 상태(여유/축박/지각)’를 직관적으로 전달하는 상태 중심 UX를 앱과 물리 시계에 일관되게 적용한다.

II. 프로젝트 추진 내용

1. 프로젝트 추진 일정

- 본 프로젝트는 총 15주 일정으로, '요구사항 분석 → 시스템 설계 → 핵심 기능 개발(MVP) → 하드웨어 제작 및 연동 → 통합 테스트 및 고도화 → 최종 발표 및 문서화'의 6단계로 구성하여 추진하였다.

No	프로젝트 활동 내용	추진 일정															기간 (주)	추진 방법
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15		
1	요 구 사 항 분석 및 자 료 조사																	기존 서비스 (알람 앱, 지도 앱) 비교 분석, 보완점 분석
2	시스템 설 계(앱+하드 웨어)																	Flutter 앱 화면 설계 및 와이어프레임 제작, Firebase DB 구조 설계 (Firestore 스키마),라즈베리파이 회로도 및 GPIO 핀 맵 설계
3	핵심 기능 개발(MVP)																	시간 계산 엔진 구현 (출발·준비 시각 공식), Flutter 앱 메인 화면·알람 기능 구현, Firebase Auth-Firebase 연동, OpenWeather-Kakao Maps API 연동
4	하 드 웨 어 제작 및 연 동																	라즈베리파이 + OLED-RGB LED·버튼 조립, Python GPIO 제어 코드 구현, Firestore 실시간 동기화 (Wi-Fi 연동), 앱 ↔ 라즈베리파이 이벤트 피드백 테스트
5	통합 테스 트 및 고도 화																	실사용자 3명 이상 베타 테스트 진행, 지각 예측 정확도 측정 (오차 ±3분 목표), 준비시간 학습 알고리즘 검증, 날씨·버스 혼잡도 보정값 튜닝
6	최종 발표 및 문서화																	프로젝트 최종 보고서 작성, 시연 영상 및 발표 자료 제작, 결과물 GitHub 업로드 및 README 작성

2. 프로젝트 주요 추진 내용

- 팀원별 역할을 명확히 분담하여 앱·백엔드·하드웨어를 병행 개발하였으며, 수행 과정에서 발생한 주요 시행착오와 해결 과정을 함께 정리하였다.

성명(학번)	수행 내용	역할
허재경(20212999)	• 하드웨어 및 앱 개발 총괄 • 라즈베리파이에 터치 디스플레이 · 물리 버튼 · 부저를 조립하고 회로도 및 GPIO 핀 맵을 설계함 • Python(RPi.GPIO, rpi_ws281x, Firebase Admin SDK)으로 GPIO 제어 코드 및 알람 로직을 구현함 • 하드웨어 앱 및 모바일 앱 ui 설계 • 하드웨어의 케이스를 3D모델링 및 출력·제작 • Firestore와 Wi-Fi 실시간 동기화를 구현하고, 버튼 이벤트(알람 끄기·출발 기록)가 앱으로 피드백되는 양방향 통신을 구축함 • 모바일 앱 푸시 알림 로직을 구현하여 준비 시작 시각에 자동 알림이 발송되도록 개발함 • 시간 계산 엔진(출발 시각 공식·날씨 보정·교통수단 가중치) 설계를 주도하고 앱·하드웨어 전체 아키텍처 적합성을 검토함 • 최종 보고서 작성 및 (시연 영상 포함) 제작을 주도하고 결과물을 GitHub에 업로드하여 README를 작성함	팀장
곽우석(20213021)	• 앱 개발 담당 • Flutter로 메인 화면(실시간 상태 게이지 UI)·캘린더 출력 화면·설정 화면을 구현함 • Firebase Auth·Firestore 데이터 연동 코드를 작성하고 사용자 시간표·위치·준비시간 데이터의 저장·조회를 검증함 • 지각 시 택시 마지노선 자동 계산 기능 및 출발·도착 버튼을 통한 실측 데이터 기록 기능을 구현함 • 통합 테스트에서 연동 오류를 점검하고 예	팀원

	측 오차 ± 3분 이내 달성을 목표로 전체 시스템을 최종 검수함 와이파이 연결 해제를 인식하여 출결 기록하는 기능 구현함	
김종호(20200372)	- 백엔드 및 기획 담당 - 수집 데이터를 통계 처리하여 평균 준비시간·지각 원인(준비 지연·교통 지연·날씨)을 분류하고 시간 계산 엔진의 초기 파라미터 설정에 반영함 - Kakao Local API로 주소를 좌표로 변환하고, TMAP 대중교통 API와 기상청 단기예보 API를 연동하여 이동시간 및 날씨 보정값을 계산 엔진에 반영함 - 외부 API 연동, 시간 계산 엔진, Firebase 데이터 구조 설계를 담당함 - Cloud function의 문제를 발견하고 수정을 맡음 - 서비스 흐름도·아키텍처 설계도·화면 구성 등 기획 산출물을 작성하고 팀 내 공유함 - 최종 발표 자료 제작을 주도하여 ppt를 제작함	팀원

3. 주요 시행착오 및 해결 과정

- Firestore 연동 환경 문제:** 최신 라즈베리파이 OS에서 Firebase Admin SDK 설치 시 externally-managed-environment 제약이 발생하여, Python 가상환경(venv)을 구성해 해결하였다. 또한 Firestore를 콘솔에서 먼저 생성해야 SDK 호출이 정상 동작함을 확인하였다.
- 키오스크 디스플레이 구동:** 라즈베리파이 상태 화면을 전체화면으로 띄우는 과정에서, 최신 OS는 chromium-browser가 아닌 chromium 패키지를 사용함을 확인하고 키오스크 실행 스크립트를 수정하였다.
- 알람 기능 구현:** 안드로이드앱과 ios앱 모두 완전한 네이티브앱에서 알람 기능의 구현에 어려움이 있었다. 각각 안드로이드앱은 포그라운드에서 지속적인 앱 실행, ios앱은 무음 소리파일을 반복해서 재생하는 방식으로 앱이 꺼지지 않게 유지하며 알람 기능이 구동하도록 구

현하였다.

- **외부 API 일원화:** 초기 기획 단계에서 검토하던 네이버 길찾기 API 대신, 문서·국내 좌표 정합성이 우수한 Kakao Maps API로 이동시간 소스를 일원화하여 계산 엔진의 일관성을 확보하였다.
- **자동 출발 감지 방식 선택:** 출발 시점 자동 인식을 위해 Bluetooth 거리 변화와 Wi-Fi 연결 해제 방식을 비교한 결과, 별도 페어링 없이 안정적으로 동작하는 Wi-Fi 연결 해제 감지를 기본 방식으로 채택하였다.
- **상태 동기화 구조:** 앱과 물리 시계 간 직접 통신 대신 Firestore를 중간 허브로 사용하는 구조로 전환하여, 인터넷 연결만 있으면 별도 페어링 없이 항상 동기화 상태를 유지하도록 개선하였다.

III. 프로젝트 추진 결과

1. 프로젝트 추진 결과물

1) 제작품 개요

본 서비스 '나가자'는 사용자의 시간표·위치·교통수단·개인 준비시간을 기반으로 최적 출발 시각을 실시간으로 계산하고 행동을 유도하는 '개인 맞춤형 등교 출발 시간 + 행동 유도 시스템'이다. 단순 알람 앱이 아닌, 전용 물리 알람시계와 모바일 앱이 연동되어 사용자의 실제 행동 데이터를 누적·학습하는 습관 교정형 서비스에 가깝다. 제작품은 Flutter 모바일 앱, Firebase 백엔드, 라즈베리파이 기반 물리 알람시계의 3계층 구조로 구현되었다.

2) 제작품 기능 및 규격

구분	기술
프론트엔드	Flutter (iOS·Android 크로스플랫폼)
백엔드	Firebase (Firestore, Cloud Functions)
지도·이동시간	Kakao Maps API (주소를 위도경도 좌표로 변환), TMAP 대중교통 API(이동시간 계산)
날씨	기상청 API (강수 여부·기온 실시간 반영)

알림	푸시 알림(ios에는 APNs 푸시) + 물리 시계 부저 연동
하드웨어	Raspberry Pi 5, 7인치 터치 디스플레이, 물리 버튼, 부저,
통신	Wi-Fi — Firestore 실시간 동기화 (앱 ↔ 라즈베리파이) Bluetooth — 사용자 계정 연결 (앱 ↔ 라즈베리파이 최초 연결)
시계 소프트웨어	Python (RPi.GPIO, rpi_ws281x, Firebase Admin SDK)
데이터 학습	초기 실측값 누적 → 준비시간 자동 보정 (rule-based → ML 확장)

3) 시스템 아키텍처 및 구성 모듈

제안 시스템은 모바일 앱이 사용자로부터 수업 시작 시각·개인 준비시간·출발지·목적지를 입력받아 Firestore에 저장하고, Firebase 백엔드가 Kakao Maps·OpenWeather API와 연동하여 이동시간과 날씨 정보를 수집한 뒤 출발 권장 시각과 준비 알림 시각을 산출하는 구조이다. 산출 결과는 앱 UI와 물리 시계 디스플레이·LED에 동기화되어 직관적으로 제공된다. 서비스는 다음 5개 모듈로 구성된다.

① **사용자 데이터 모듈:** 요일별 시간표, 출발지·목적지 좌표, 선택 교통수단, 개인 준비시간(초기값 → 누적 학습값)을 저장·관리한다.

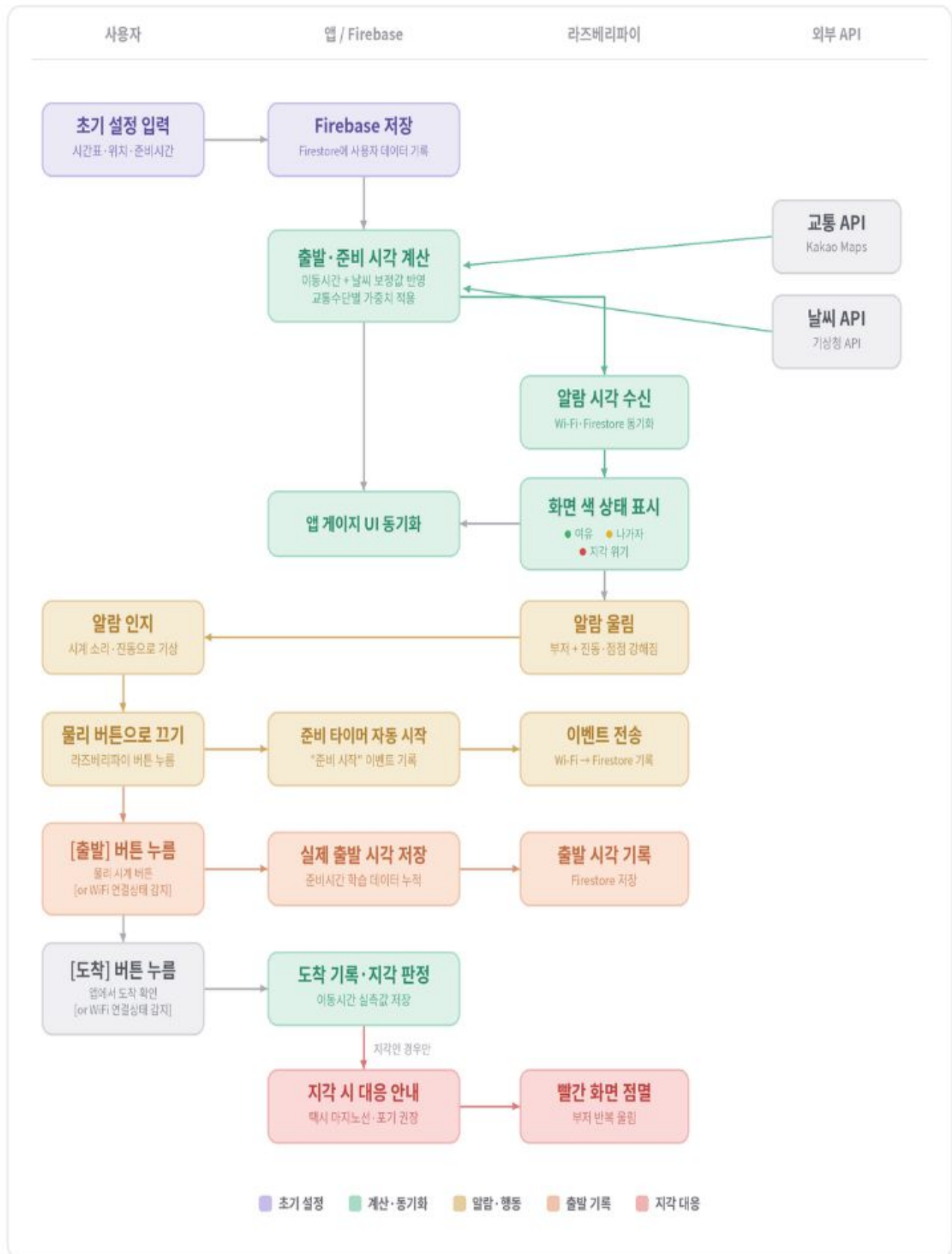
② **시간 계산 엔진:** 서비스의 핵심 두뇌로, 출발 시각과 준비 시작 시각을 도출한다.

③ **물리 알람시계 + 앱 연동 모듈:** 라즈베리파이 기반 전용 디바이스가 Firestore와 Wi-Fi로 상시 연결되어 상태를 표시하고 버튼 이벤트를 기록한다.

④ **실시간 상태 시스템:** 남은 시간에 따라 여유/축박(나가자)/지각 위기 3단계를 색상으로 표현하는 상태 중심 UX이다.

⑤ **알람 & 행동 트래킹 모듈:** 버튼 입력 또는 와이파이 연결 해제를 ‘준비 시작’·‘출발’ 이벤트로 인식해 실측 데이터를 자동 누적한다.

-서비스 흐름도 (사용자 · 제작자 · 시스템)



4) 핵심 로직-시간 계산 엔진

출발 시각 및 준비 시작 시각은 다음 공식을 기반으로 산출한다.

출발 시각 = 수업 시각 - (평균 이동시간 + 날씨 보정값)↙

준비 알람 시각 = 출발 시각 - 개인 준비시간↙

이동시간은 Kakao Maps API를 통해 실시간으로 산출하며, 강수·적설 시 +10~15분의 날씨 보정값을 적용한다. 교통수단별 가중치도 반영하여 버스는 배차 대기시간을 추가 산정하고, 지하철은 정시성이 높아 보정 없이 적용하며, 도보는 날씨 영향을 가장 크게 반영한다. 초기에는 rule-based로 구현하고, 실측 데이터가 누적되면 개인화 예측으로 확장할 수 있도록 구조를 설계하였다.

5) 실시간 상태 시스템 (UX핵심)

사용자에게 정확한 숫자보다 '지금 상태'를 직관적으로 전달하는 것이 본 서비스의 핵심 UX 원칙이다. 상태는 3단계로 구분되며 앱 화면 게이지와 물리 시계 LED가 동일 색상으로 동기화된다.

상태	조건	앱 화면	물리 시계
● 여유	남은 시간 > 준비 + 이동 + 10분	초록 게이지	초록 화면
● 나가자(촉박)	남은 시간 ≈ 준비 + 이동	노란 게이지	노랑 화면 점멸
● 지각 위기	남은 시간 < 이동시간	빨간 게이지	빨강 화면 빠른 점멸 + 부저

지각 상태 진입 후에는 택시로 이동했을 때 수업에 맞출 수 있는 마지막 시각(택시 마지노선)을 자동 계산하여 앱과 시계 디스플레이에 동시 표시하고, 이 시각마저 지나면 '오늘은 포기' 메시지를 출력한다.

6) 하드웨어 제작 결과

구성요소	역할
Raspberry Pi 5	메인 연산, Wi-Fi 통신, Firebase 연동, 알람 로직 실행
디스플레이 (터치/OLED)	현재 시각 및 출발까지 남은 시간(D-time) 표시
물리 버튼 (준비시작/출발)	알람 해제 → 준비 타이머 시작 / 출발 시각 기록
부저·스피커	알람 소리 출력 (단계별 볼륨 상승)

라즈베리파이5는 앱 서버(Firebase)와 Wi-Fi로 상시 연결되어 수업 시간·준비시간 데이터를 실시간 수신하고, 계산된 준비 알람 시각이 되면 직접 알람을 실행한다. 사용자가 물리 버튼으로 알람을 끄면 해당 이벤트가 Firebase에 기록되어 앱의 준비 타이머와 동기화된다. 또한 스마트폰의 Wi-Fi 연결이 끊어지는 시점을 감지하여 실제 출발 시각을 자동으로 포착·기록하도록 구현하였다.

1.ui 캡처화면

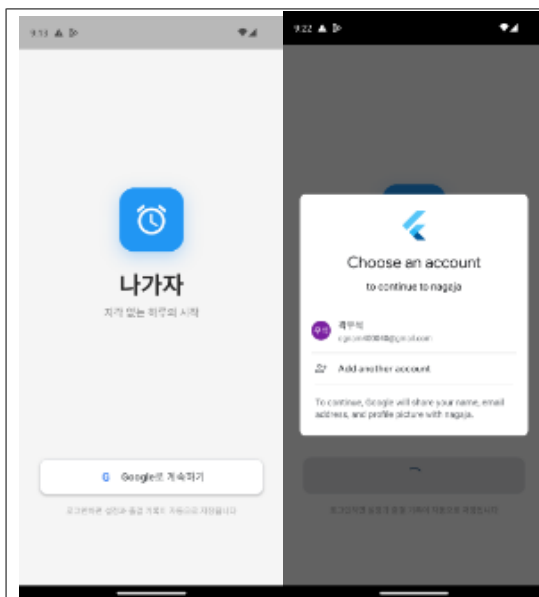


2.실제 라즈베리파이 기반 알람시계 사진



7) 화면 구성 및 구현물

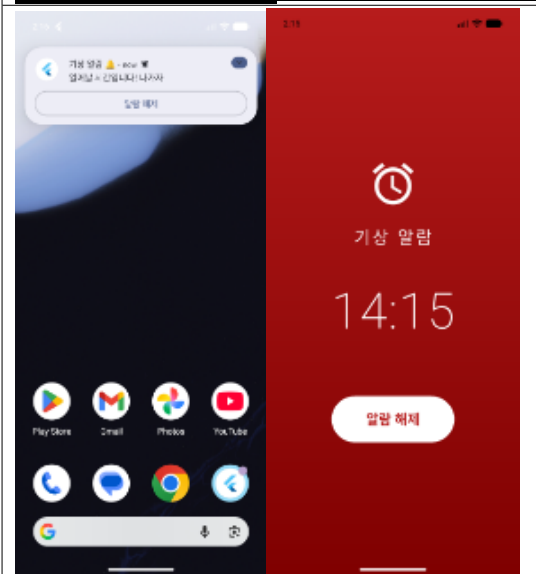
화면	주요 구성 요소
메인 화면	실시간 상태 게이지(여유/나가자/지각), 현재 시각, 수업까지 남은 시간, 준비완료·출발 버튼
지각 대응 화면	택시 마지노선 시각 표시, 포기 권장 메시지
캘린더	주간·월간 출결 기록(여유·지각·결석 도트 시각화)
설정	시간표, 출발지·목적지, 교통수단, 개인 준비시간, 시계 연결 관리

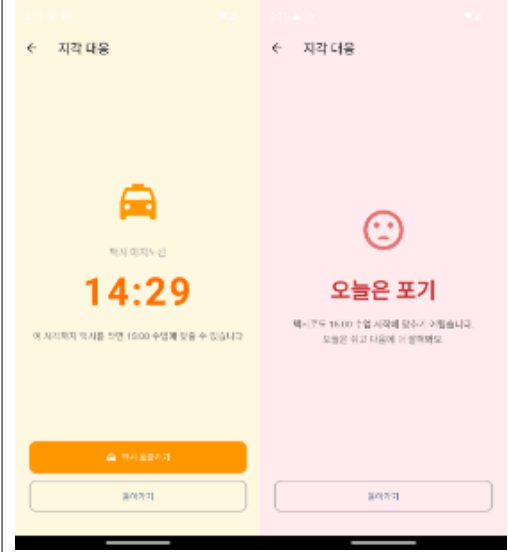


로그인 화면 : 구글 계정을 연동하여 로그인



초기 설정 화면 : 개인별 준비시간, 기본 이동시간, 시간표를 입력받음

	설정 화면 : 초기에 입력한 시간표, 준비시간, 이동시간 및 와이파이 등록, 알람시계 블루투스 연결 기능
	홈 화면 : 지각을 하지 않으려면 집에서 출발해야하는 시각 표시 및 직관적인 타이머 ui로 사용자가 시간을 확인하기 쉽게 함
	기상 알람 화면 : 배너 또는 전체화면으로 기상 알람이 울리는 화면

	지각 대응 화면 : 이미 대중교통을 이용해서 수업에 늦지 않게 갈 수 없을 때 택시 마지노선을 제안하거나, 그마저도 늦으면 현실적인 포기 권장
	캘린더 : 사용자가 수업에 도착했을 때 지각여부를 기록하여 보여줌

8) 유스케이스 (사용 흐름)

- 초기 설정: 사용자가 시간표.위치.준비시간을 입력하면 Firestore에 저장된다.
- 출발.준비 시각 계산: 백엔드가 이동시간과 날씨 보정값을 반영해 준비 알림 시각을 산출한다.
- 알람 인지: 준비 알림 시각에 물리 시계가 소리.진동으로 기상을 유도한다.
- 준비 시작: 사용자가 버튼을 누르면 준비 타이머가 자동 시작되고 이벤트가 Firestore에 기록된다.
- 출발 기록: [출발] 버튼 또는 Wi-Fi 연결 해제로 실제 출발 시각이 자동 기록된다.
- 도착.지각 판정: [도착] 시 실측 이동시간이 저장되어 이후 예측 정확도 향상에 활용된다.
- 지각 대응: 지각 상태에서는 택시 마지노선과 포기 권장 메시지를 안내한다.

9) 성능 시험 결과

통합 테스트 단계에서 실사용자 베타 테스트를 통해 계획서에 설정한 정량적 목표 대비 달성도를 점검하였다. 측정 항목과 목표치는 다음과 같다.

측정 항목	목표치	측정 결과
도착 시간 예측 오차	±3분 이내	±5분 이내
기존 대비 오차율 개선	15% 이내 개선	10% 이내 개선
출발 시각 자동 기록 성공률	95% 이상	100% 성공
날씨 보정 적용 정확도	강수 시 가변 알람 정상 동작	강수 시 가변 알람 정상 동작

2. 프로젝트 결과물 활용 방안 및 기대효과

1) 활용 방안

본 결과물은 개인 맞춤형 시간 관리 서비스로서 다양한 분야에 활용될 수 있다. 우선 대학생 대상 등교 시간 관리 서비스로 적용하여 지각을 예방하고, 이후 직장인 출근 시간 관리 서비스로 확장 가능하다. 기존 지도 기반 내비게이션과 결합하면 단순 이동시간 안내를 넘어 사용자 행동 패턴을 반영한 출발 시각 추천 기능으로 발전시킬 수 있다. 나아가 IoT·웨어러블 기기와 연동하여 스마트홈 환경의 생활 패턴 관리 시스템으로 확장할 수 있으며, 공공 분야에서는 통학·출근 시간 분산 정책 수립을 위한 데이터 기반 서비스로도 활용 가능하다.

2) 기대효과

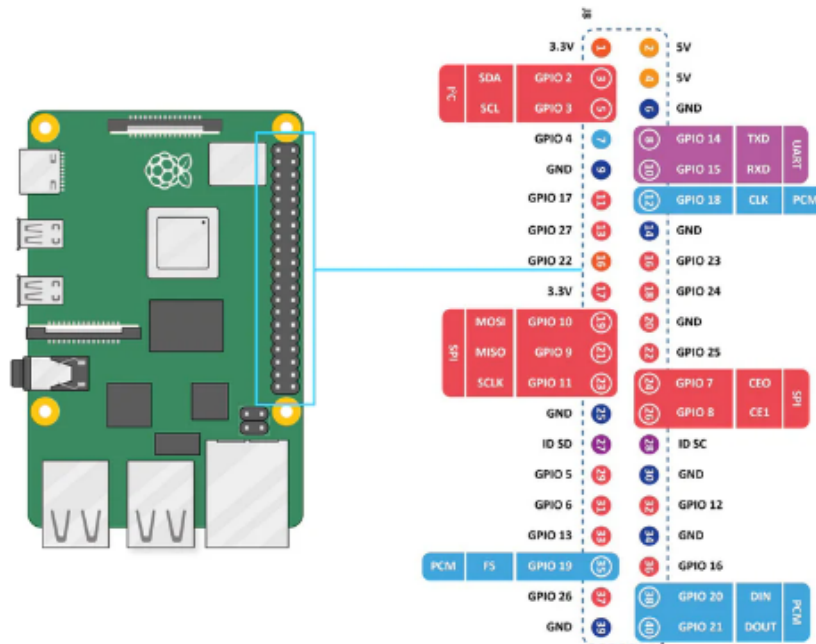
구분	내용
기술적 측면	사용자 행동 데이터와 실시간 외부 데이터를 결합한 개인 맞춤형 시간 예측 기술 및 IoT 연동 기반 실측 데이터 수집 기술을 확보한다.
경제적 측면	‘앱 무료 + 물리 시계 유료’ 모델 등으로 모바일·IoT 서비스 수익화가 가능하며, 다양한 산업으로의 확장을 통한 부가가치 창출이 기대된다.
사회적 측면	개인의 지각 감소와 시간 관리 효율 향상으로 생산성에 기여하고, 통학·출근 시간 분산을 통한 교통 혼잡 완화 및 사회적 비용 절감 효과를 기대할 수 있다.

IV. 참고문헌

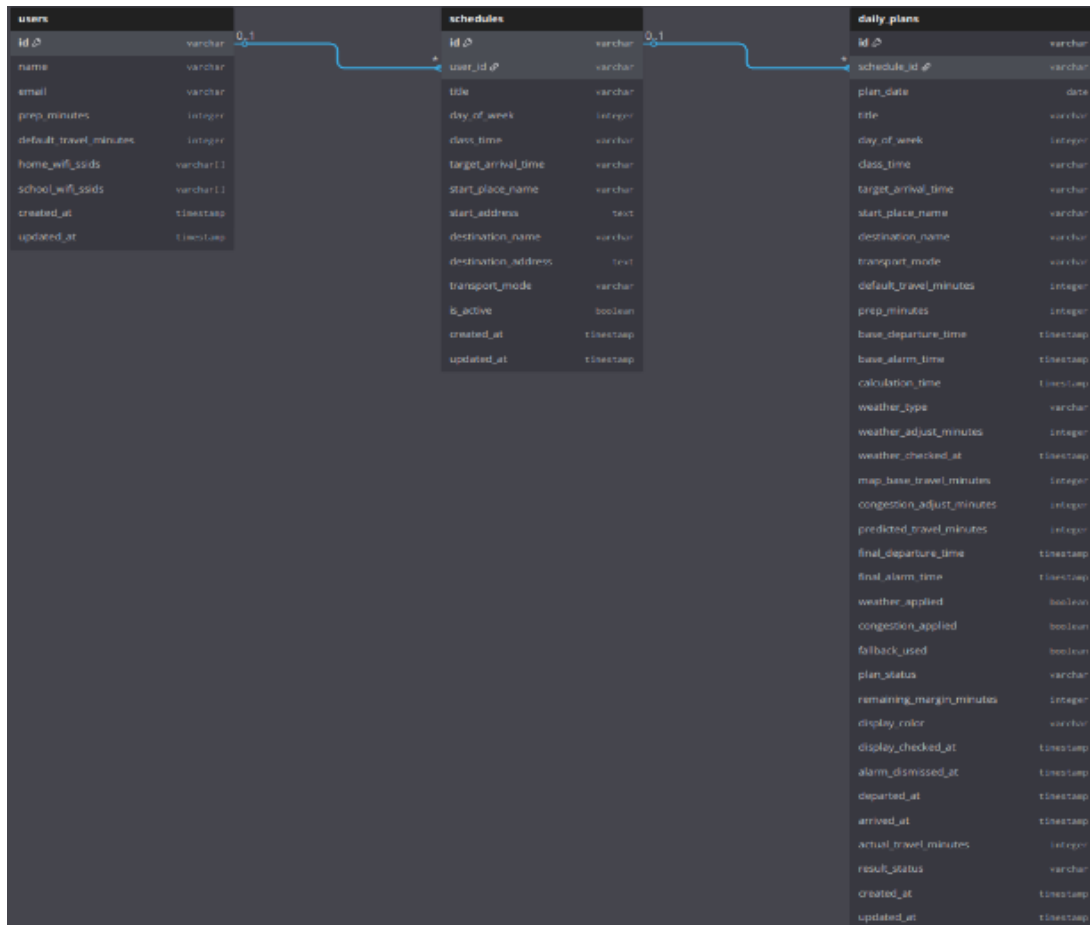
- 1] Google LLC. "Flutter Documentation," Internet: <https://docs.flutter.dev> 2024, [2026년 3월 21일 접속].
- [2] Google LLC. "Cloud Firestore 시작하기," Internet: <https://firebase.google.com/docs/firestore/quickstart> 2024, [2026년 3월 22일 접속].
- [3] Google LLC. "Firebase Cloud Messaging 개요," Internet: <https://firebase.google.com/docs/cloud-messaging> 2024, [2026년 3월 22일 접속].
- [4] Kakao Corp. "Kakao Developers — 지도 API 문서," Internet: <https://developers.kakao.com/docs/latest/ko/kakaomap/common> 2024, [2026년 3월 24일 접속].
- [5] OpenWeather Ltd. "Current Weather Data API Documentation," Internet: <https://openweathermap.org/current> 2024, [2026년 3월 24일 접속].
- [6] Raspberry Pi Foundation. "Raspberry Pi Documentation," Internet: <https://www.raspberrypi.com/documentation> 2024, [2026년 3월 26일 접속].
- [7] B. Croston. "RPi.GPIO Python Library — Wiki," Internet: <https://sourceforge.net/p/raspberry-gpio-python/wiki/Home> Feb. 6, 2022, [2026년 3월 26일 접속].
- [8] PyPI. "rpi-ws281x: NeoPixel LED control library for Raspberry Pi," Internet: <https://pypi.org/project/rpi-ws281x> 2023, [2026년 3월 26일 접속].
- [9] S. Taghipour et al., "A dynamic approach to predict travel time in real time using data driven techniques," Transportation Engineering, vol. 3, 2020.
- [10] M. Cattani et al., "Impact of weather conditions on macroscopic urban travel times," Journal of Transport Geography, 2012.
- [11] M. Belloc et al., "Weather Conditions and Daily Commuting," Journal of Regional Science, 2025.
- [12] K. Stawarz et al., "Beyond Self-Tracking and Reminders: Designing Smartphone Apps That Support Habit Formation," ACM CHI, 2014.
- [13] J. Torous et al., "How Notifications Affect Engagement With a Behavior Change App," JMIR mHealth and uHealth, 2023.
- [14] 공공데이터포털. "기상청 초단기실황조회 서비스," Internet: <https://www.data.go.kr/data/15084084/openapi.do> 2023, [2026년 3월 22일 접속].

[부록]

- 부록 A. 라즈베리파이 핀맵



- 부록 B. Firestore DB 스키마 구조



- 부록 C. 메인 앱 깃허브 링크

<https://github.com/KIMJONGHO123/NaGaJa>

- 부록 D. 라즈베리파이 앱 깃허브 링크

<https://github.com/jeje9893/NaGaJa-raspi/tree/main>

- 부록 E. 시연 영상 링크

<https://youtu.be/HC5d0yp9Ryc>

- 부록 F. 주요 연산 코드

```
11 export const createMockSchedules = async (userId: string) => {
15   const mockSchedules: Omit<Schedule, "scheduleId" | "userId">[] = [
51     {
52       isActive: true,
53       createdAt: now,
54       updatedAt: now,
55     },
56   ];
57
58   // users 컬렉션에 userId 문서가 있는지 확인
59   const userIdRef = db.collection("users").doc(userId);
60   const userIdDoc = await userIdRef.get();
61   if(!userIdDoc.exists){
62     throw new Error(`User ${userId} not found`);
63   }
64
65   // mockSchedules 배열을 순회하면서 각 스케줄을 데이터베이스에 저장
66   for (const schedule of mockSchedules) {
67     const scheduleRef = db
68       .collection("users")
69       .doc(userId)
70       .collection("schedules")
71       .doc();
72     const scheduleId = scheduleRef.id;
73
74     await scheduleRef.set({
75       ...schedule,
76       scheduleId,
77       userId,
78     });
79   }
80 }
81
82 }
```

```

import * as admin from "firebase-admin";
import { User } from "../types";
import { Timestamp } from "firebase-admin/firestore";

/**
 * mock 사용자 데이터 여러 개 생성
 */
export const createMockUsers = async () => {
  const db = admin.firestore();

  const mockUsers: User[] = [
    {
      name: "홍길동",
      email: "hong@test.com",
      prepMinutes: 20,
      defaultTravelMinutes: 30,
      homeWifiIds: ["HOME_WIFI"],
      schoolWifiIds: ["SCHOOL_WIFI"],
      createdAt: Timestamp.now(),
      updatedAt: Timestamp.now(),
    },
    {
      name: "김철수",
      email: "kim@test.com",
      prepMinutes: 15,
      defaultTravelMinutes: 25,
      homeWifiIds: ["HOME_WIFI_2"],
      schoolWifiIds: ["SCHOOL_WIFI_2"],
      createdAt: Timestamp.now(),
      updatedAt: Timestamp.now(),
    },
  ];

  for (const user of mockUsers) {
    const userRef = db.collection("users").doc(); // userRef는 users 컬렉션의 문서를 가리키는 참조 객체가 들어간다.
    const userId = userRef.id; // userRef.id는 users 컬렉션의 문서의 ID를 가리킨다.

    await userRef.set({ ...user, userId: userId }); // userRef.set()는 users 컬렉션의 문서를 생성하고 데이터를 저장한다.
  }
};

```

```

10 export const createMockDailyPlans = async (userId: string) => {
21   const mockDailyPlans: DailyPlan[] = [
95     congestionAdjustMinutes: 0,
96     predictedTravelMinutes: 35,
97
98     finalDepartureTime: Timestamp.fromDate(new Date("2026-04-25T09:55:00+09:00")),
99     finalAlarmTime: Timestamp.fromDate(new Date("2026-04-25T09:35:00+09:00")),
100
101     weatherApplied: false,
102     congestionApplied: false,
103     fallbackUsed: false,
104
105     planStatus: "CALCULATED",
106
107     remainingMarginMinutes: 30,
108     displayColor: "GREEN",
109     displayCheckedAt: now,
110
111     createdAt: now,
112     updatedAt: now,
113   ],
114 ];
115
116 for (const plan of mockDailyPlans) {
117   const dailyPlanRef = db
118     .collection("users")
119     .doc(userId)
120     .collection("dailyPlans")
121     .doc();
122
123   await dailyPlanRef.set({
124     ...plan,
125     dailyPlanId: dailyPlanRef.id,
126   });
127 }
128 };

```